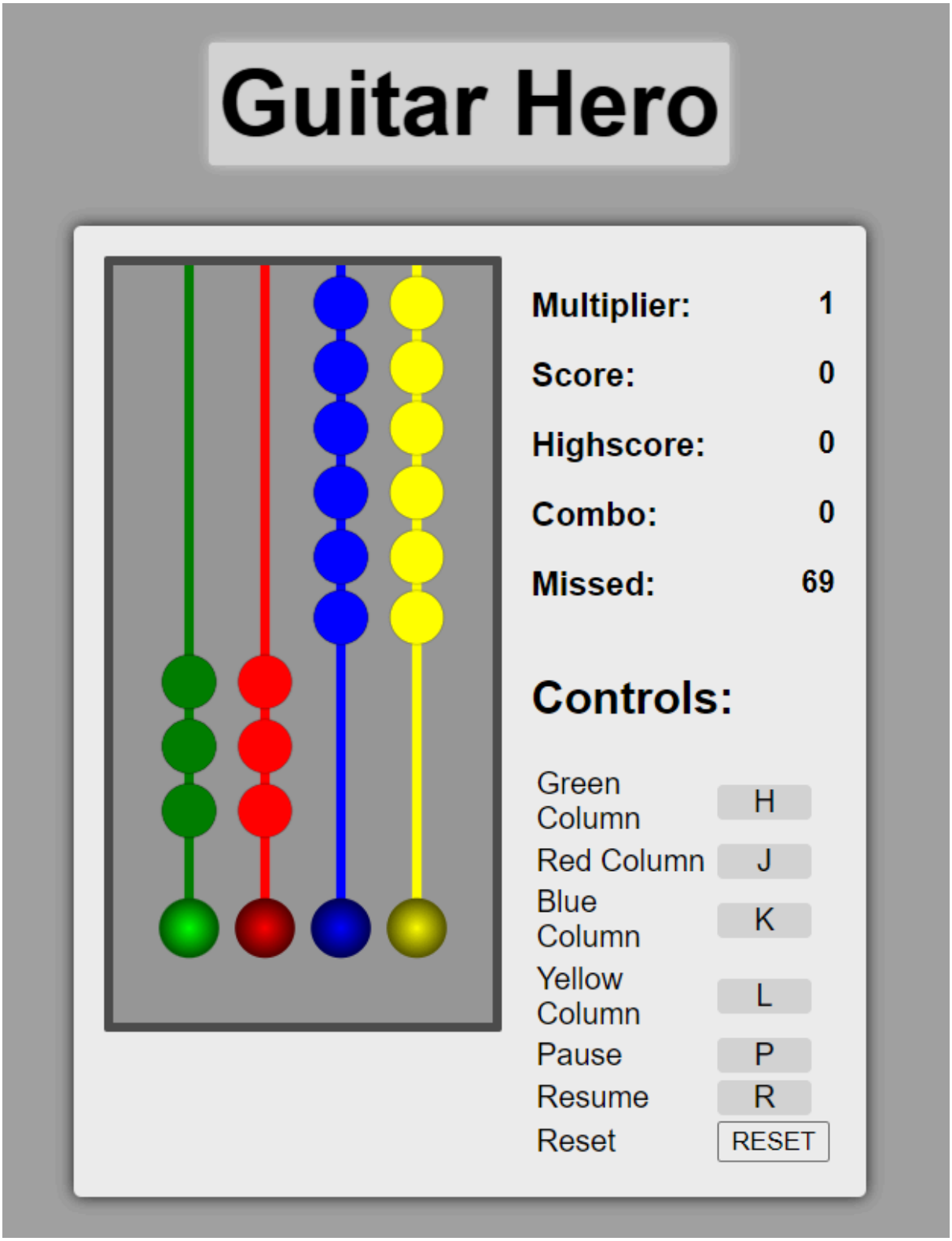
FIT2102 Assignment 1 Report
Guitar Hero
Jack Lim Jie Yang 34330062



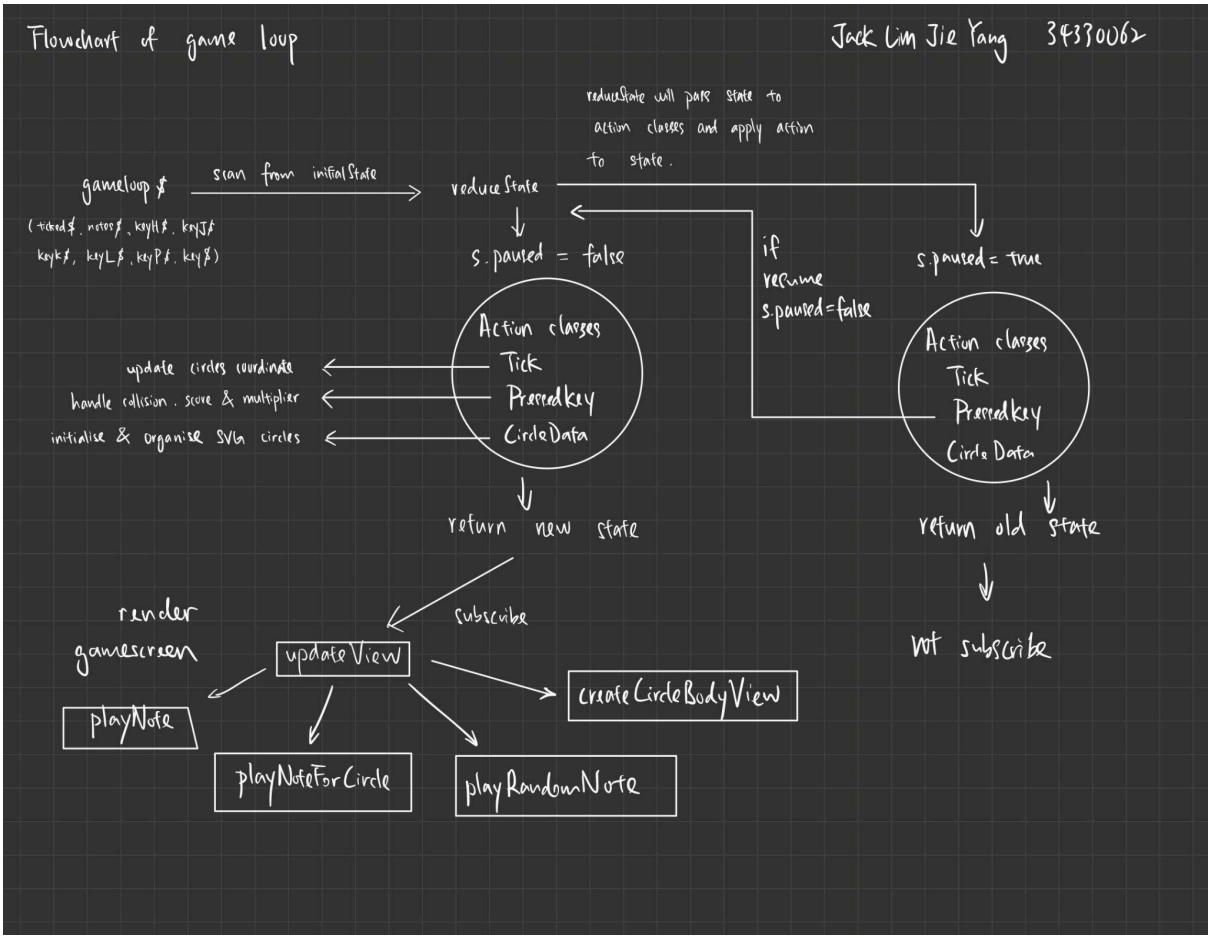
Overview:

- Functions are broken down into small functions such that the overall program is readable, independent of each other and also reusable in certain scenario (FRP style programming)
- Using ternary operators are mostly used to ensure declarative, however if else statements are also used when it comes to complex logic (easier understanding)
- Using HOF in updateView when render the gamescreen (pass a function to another function) (65 words)

Design Decisions:

- main.ts: handles the core game loop logic
- types.ts: handles the type definitions
- util.ts: contains utility functions (parsing csv into a stream of observables)
- state.ts: manages state processing and transformations
- view.ts: render the gamescreen (38 words)

Rough overview of GameLoop



main.ts

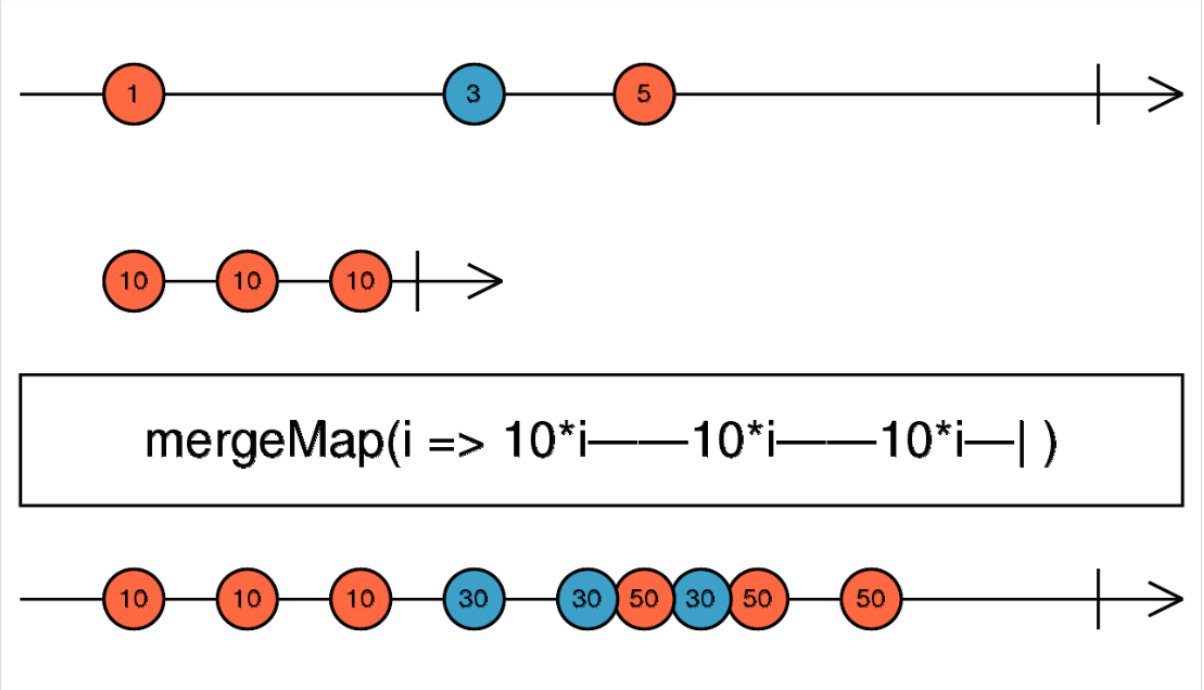
- Using observables (fromEvent, fromKey, …) from RxJS to create event streams for different actions in game. Ex: key press using (fromKey)
- Merge these observable streams into a single stream that emits data based on time continuously
- Scan to apply the reduceState function to initialState to accumulate changes of state over time.
- Subscribe to render the gamescreen based on the state given continuously.

(62 words)

We can easily add new stream such as another key control using fromEvent and reduce, subscribe it without modifying the existing code and streams. Thus, it is flexible to extend and easy to debug. (34 words)

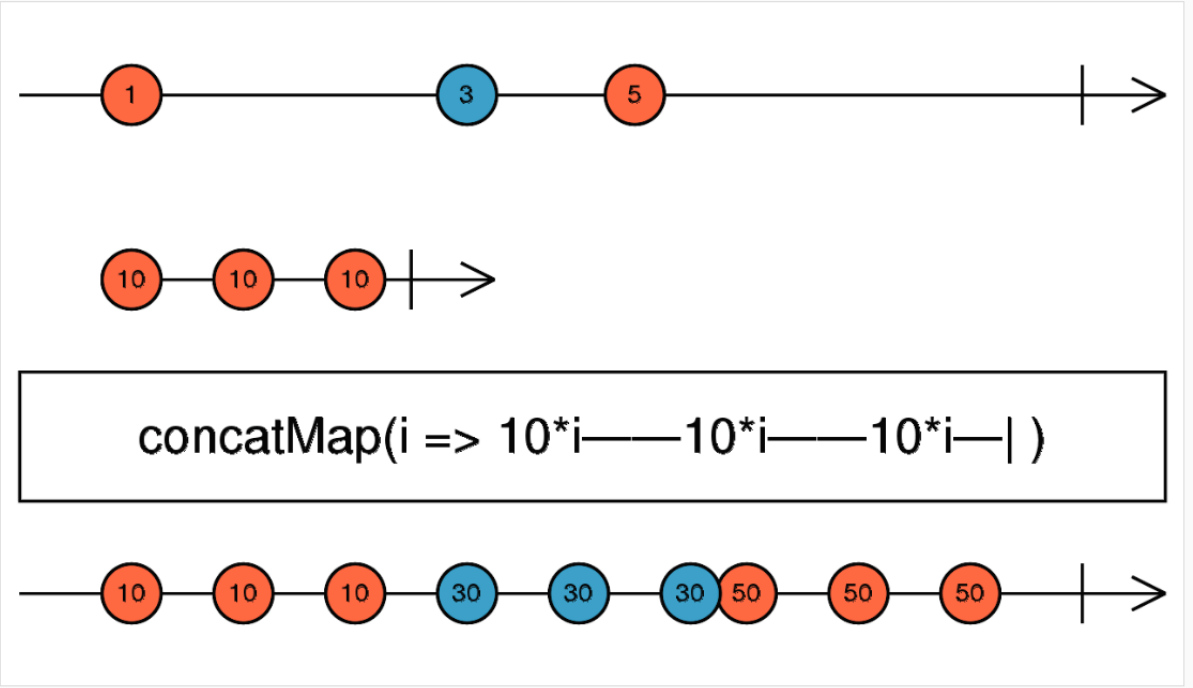
Interesting point: mergeMap and concatMap in parsing CSV file:

mergeMap:



mergeMap subscribes observables once it is emitted, multiple observables can run at the same time (concurrently). (16 words)

concatMap:



concatMap subscribes observable one at a time, the former one must be completed before processing latest one. This works well for contents arranged in ascending and the timing doesn't overlap inside the CSV.

However, we can have multiple notes appear at the same time (concurrent), using mergeMap is a better implementation to process CSV. (54words)

State management in state.ts:

- reduceState is a state transducer that will constantly apply event or action to the state for every observables merged in the game loop, starting from initialState.

- Actions classes (Tick, CircleData, PressedKey) implement action interface and only handle their respective state updating when reduceState is being accumulated. (from Asteroids)

- Ex: PressedKey class only handles logic for key press, checking collisions, calculating score increments, it is encapsulated by itself, similar to the Tick and CircleData class.

- reduceState will takes in actions from Action class, initialState (at beginning) and constantly apply action to the state using scan (accumulate updated states)

- Spread operators are used here to create copy of the existing state while updating the new state. This ensures that the functions remain pure and not mutating as we update the state.

Using this design, we can flexible update the game with new feature by creating a new class that implements the Action interface and apply the return a new state based on old state without modifying the existing code in the future. (pure function)
172 words

type.ts

- Types are declared as ReadOnly and constant to ensure immutability when pass into different functions. 15 words

view.ts

- Render the contents stored in state to gamescreen, including background-music and user-played notes, random notes, circle rendering, score, and game attributes update, which are impure.

- HOF: Using forEach method to take functions as argument and applies to each element in the respective array. 43 words

Highlight of view.ts:

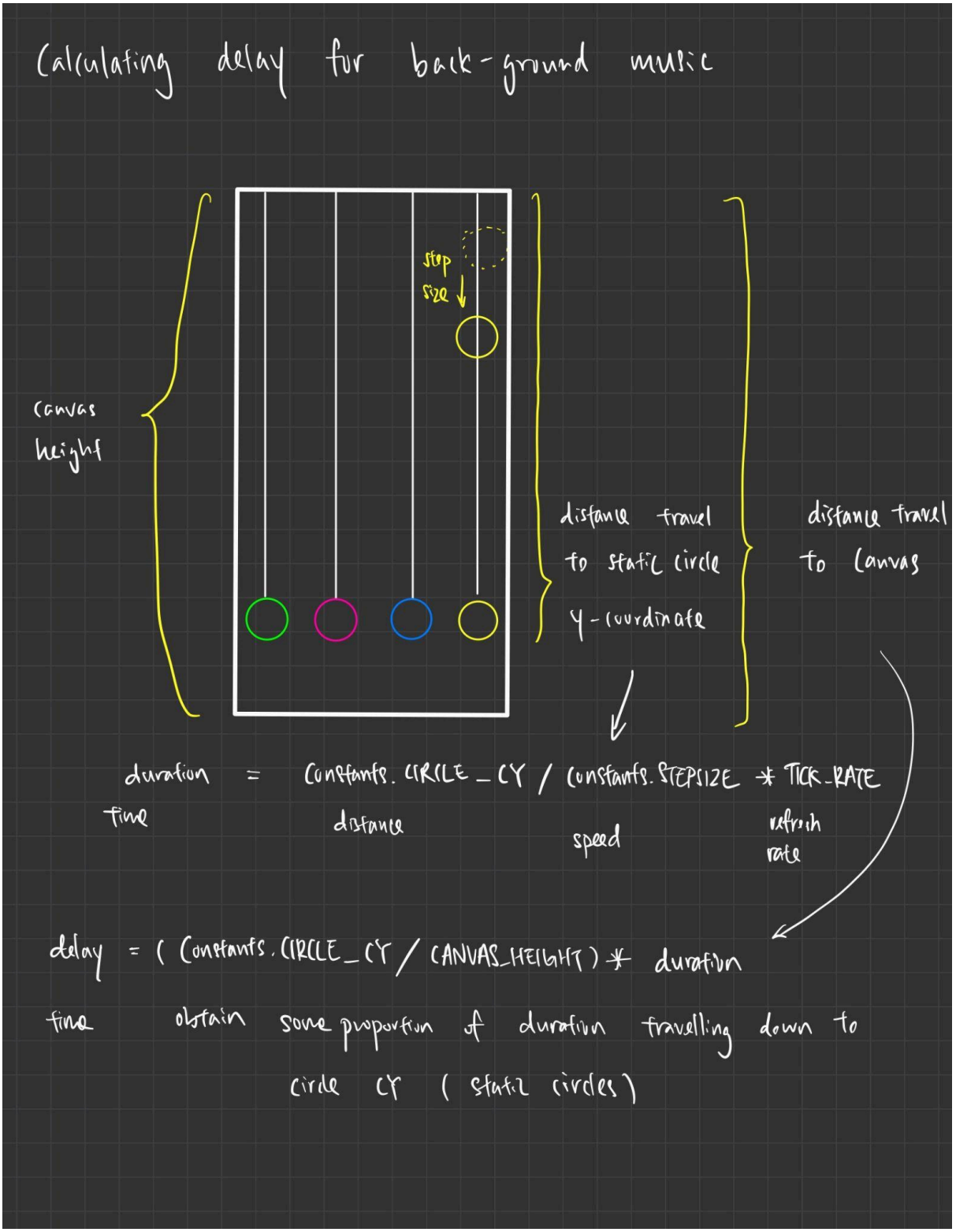
- To synchronize the audio with the visual note's arrival at the hit box, the note should be played only when the circle reaches the hit box.

- The delay before playing the note is proportional to the time it takes for the circle to reach the hit box.

- Find the duration to reach the hitbox: Constants.CIRCLE_CY / Constants.STEPSIZE, then multiply the Constants.TICK_RATE_MS

- Then delay is calculated based on the the circle's current position relative to the total height of the canvas (Constants.CIRCLE_CY / Viewport.CANVAS_HEIGHT) multiplied by the duration.

- Apply delay using RxJS **setTimeOut**, this will delay to play note after this duration of calculated delay, making it synchronous.
  110 words



Total: 609 words

Additional Features (Total 5):

1. Pause and Resume

- Added a new paused boolean attribute in state, default is false
- Indicated by keyP and keyR, created streams for each respective key and declared respective id in index.html
- Merge into gameloop$, once keyP is pressed, action class PressedKey's getStatus will be invoked, return a new state with paused set to true
- Other action class (Tick and CreateCircle) will not apply action to update the state, they keep returning old state.
- In subscribe, as the state's paused attribute is true, no rendering action executed. Hence, the game is paused.
- When keyR is pressed, action class PressedKey's getStatus will be invoked, return a new state with paused set to false. The action classes will apply action on current state then return a new state.
- In subscribe, as the state's paused attribute is false, rendering action executed with new state. Hence, the game is resumed.

This can effectively control game state, by keep returning old state when paused, states will not mutate.

2. Reset

- Created HTMLButtonElement with id resetButton in index.HTML
- Using fromEvent to keep track of clicking event for resetButton.
- If clicked, the gameloop$ will end using unsubscribe()
- The SVGcircles element on gamescreen canvas was cleared except for the bottom static circles
- The reset$ stream is then unsubscribed, then invoke the main() again to rerun the gameloop$

This can effectively reset the game using a reset button without refresh the whole page.

3. Missed Note

- Added missCount in state to keep track missing notes.
- User miss a note when :

  1) press a key when there is no note,
     - Using boolean indicator isHit, key is pressed but userPlayCircles length === 0 (when there is no note) (we get the new state from pressCircle)

     - If isHit is false, meaning pressing no circles.

  2) the circles travel out of the canvas border
     - Action class Tick handle the circle movement.
     - If there are circles expired (moving out of the canvas), we will update the missedNote here using expiredCircles.length

  Increment the missCount during these two scenario.

4. Combo and Multiplier system

- Added attributes comboCount and multiplier in state.

- In Action PressedKey class, pressCircle return a new state when user press a key.

- If it there was collision happen, concat the the collided circles into userPlayCircle to play the sound at subscribe

- Using the new state returned by pressCircle in apply, isHit will return true if the user successfully pressed on circle, comboCount increment by 1, else false mean the user missed circle, reset the comboCount to 0.

- Multiplier will be the minimum of initial multiplier and increment of 0.25 every 10 consecutive successful press.

- Render to the gamescreen in subscribe

Clearly keep track user combo and current multiplier.

5. Highscore, Score system

- Added highscore and score attributes in state.

- The score will be updated multiply with multiplier

- The highscore will be the maximum between old state's highscore and current score, only be updated when current highscore exceeds old highscore.

- The updated score and multiplier will be displayed in subscribe every tick rate

- Render to the gamescreen in subscribe

Clearly keep track current highscore and score.